CAHIER DE CHARGE

JEU DES ALLUMETTES

Projet : Développement d'application client lourd

Date : 05 mars 2026

Rédigé par : MVE Pascal Keyne

TABLE DES MATIÈRES

1. Présentation Générale 3

   1.1 Contexte et Justification du Projet

   1.2 Objectifs Pédagogiques

   1.3 Périmètre du Projet

2. Présentation Générale de l'Application 4

   2.1 Intitulé du Projet

   2.2 Description

   2.3 Utilisateurs

3. Expression des Besoins 5

   3.1 Besoins Fonctionnels

   3.2 Besoins Non Fonctionnels

4. Contraintes Techniques 6

   4.1 Technologies Imposées

   4.2 Architecture

5. Sécurité 6
6. Modélisation UML 6- 7

1. Présentation Générale

1.1 Contexte et Justification du Projet

Dans le cadre de la formation BTS SIO Option SLAM, les étudiants doivent concevoir et développer une application client lourd simulant un jeu simple avec persistance des données.

Le jeu des allumettes est un jeu classique à deux joueurs, traditionnellement joué avec des objets physiques. Cette version numérique permet de simuler le jeu sur un ordinateur, en ajoutant une traçabilité des parties via une base de données. La gestion manuelle des scores (papier, mémoire) entraîne des oublis et un manque de suivi historique.

La mise en place d'une application client lourd dédiée est donc justifiée pour automatiser le jeu, enregistrer les résultats et offrir un accès aux scores pour les joueurs.

1.2 Objectifs Pédagogiques

Le projet doit permettre à l'étudiant de :

- Analyser un besoin ludique et technique
- Rédiger un cahier des charges fonctionnel
- Concevoir l'application à l'aide de diagrammes UML
- Créer et exploiter une base de données relationnelle
- Développer une application client lourd en C#
- Appliquer les bonnes pratiques de sécurité

1.3 Périmètre du Projet

Le projet couvre :

- La logique complète du jeu des allumettes
- La gestion des joueurs et des parties
- Le suivi des scores et historique via base de données
- L'interface graphique simple

Sont exclus (hors périmètre) :

- Mode multijoueur en réseau
- Intelligence artificielle pour un mode solo
- Intégration avec des API externes
- Développement d'une version mobile ou web

2. Présentation Générale de l'Application

2.1 Intitulé du Projet

Application Client Lourd de Jeu des Allumettes

2.2 Description

L'application permet de jouer au jeu des allumettes à deux joueurs sur un même poste, avec enregistrement des résultats.

- Début avec 20 allumettes
- Chaque joueur prend 1 à 3 allumettes à tour de rôle
- Le joueur prenant la dernière allumette perd
- Enregistrement automatique des parties pour suivre les scores

2.3 Utilisateurs

- Joueur 1 et Joueur 2 : participants au jeu, saisissent leurs noms et jouent
- Administrateur (optionnel) : consultation des scores globaux

3. Expression des Besoins

3.1 Besoins Fonctionnels

Fonctionnalités générales

- Saisie des noms des joueurs
- Affichage graphique des allumettes
- Gestion des tours de jeu
- Détection de la fin de partie et du perdant

Gestion des parties

- Démarrer une nouvelle partie
- Prendre 1, 2 ou 3 allumettes
- Afficher le nombre restant
- Enregistrer la partie en base de données (vainqueur, date, etc.)

Gestion des scores

- Consulter l'historique des parties
- Afficher le classement des victoires par joueur

3.2 Besoins Non Fonctionnels

- Sécurité : protection des données en base, gestion des exceptions
- Ergonomie : interface intuitive, responsive à la fenêtre
- Performance : temps de réponse rapide (< 1 seconde pour les actions)
- Maintenabilité : code clair, organisé (architecture en couches)
- Disponibilité : application fonctionnelle en local (Windows)

4. Contraintes Techniques

4.1 Technologies Imposées

- Langage : C# (.NET)
- Base de données : MySQL (PhpMyadmin)
- Frontend : Windows Forms ou WPF
- Environnement de développement : Visual Studio

4.2 Architecture

- Architecture en couches (UI / Logique / Accès Données) recommandée
- Organisation claire du code (séparation présentation / métier / données)

5. Sécurité

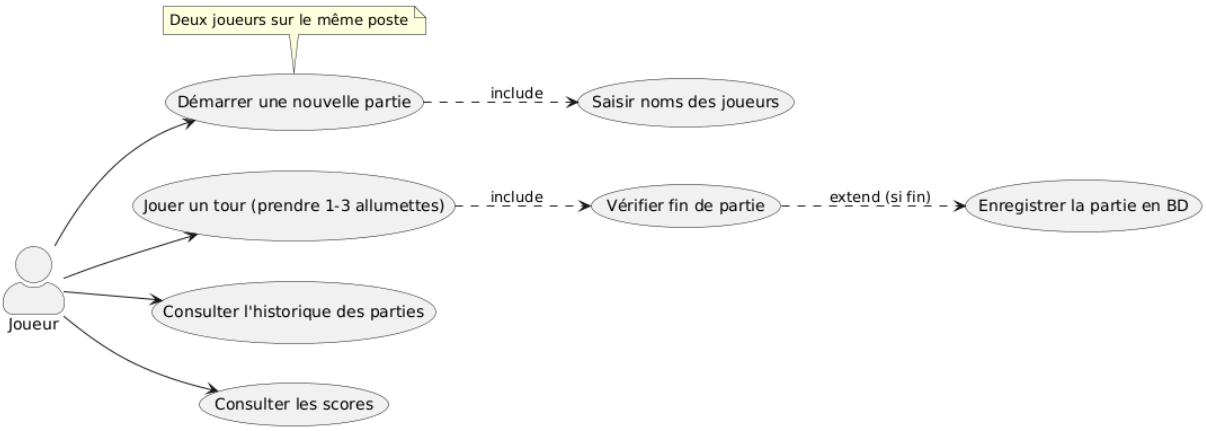
- Validation des entrées utilisateur
- Utilisation de commandes paramétrées (SqlCommand)
- Gestion des exceptions pour éviter les crashes
- Accès restreint à la base de données

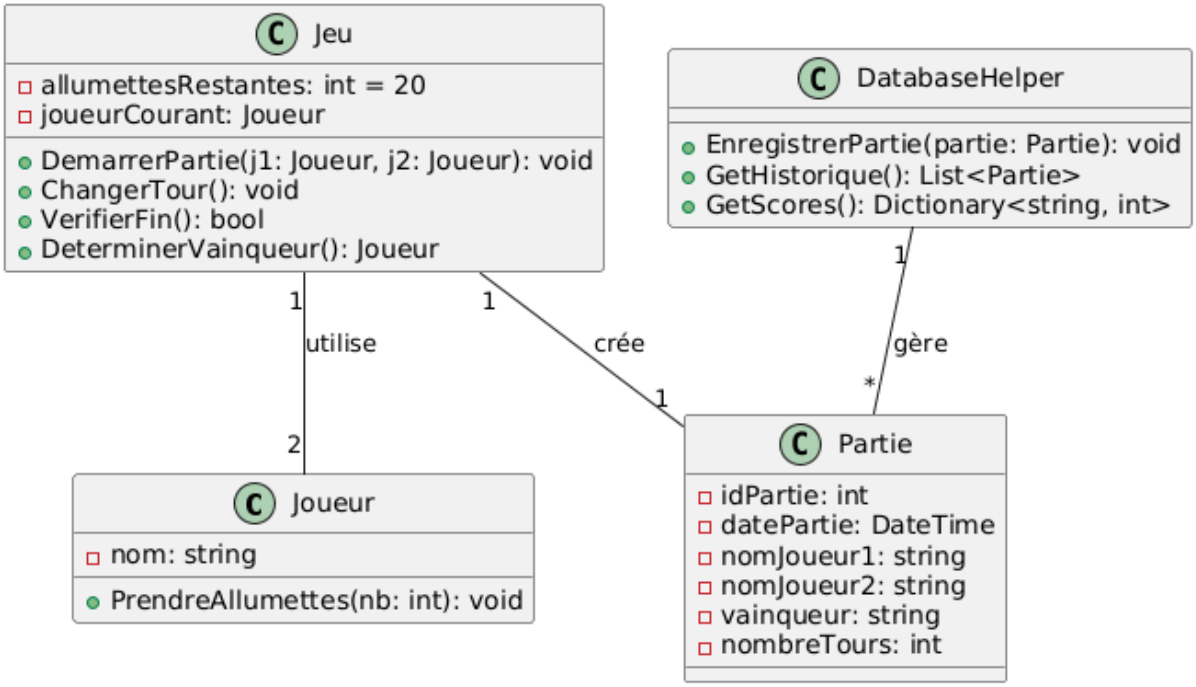
6. Modélisation UML

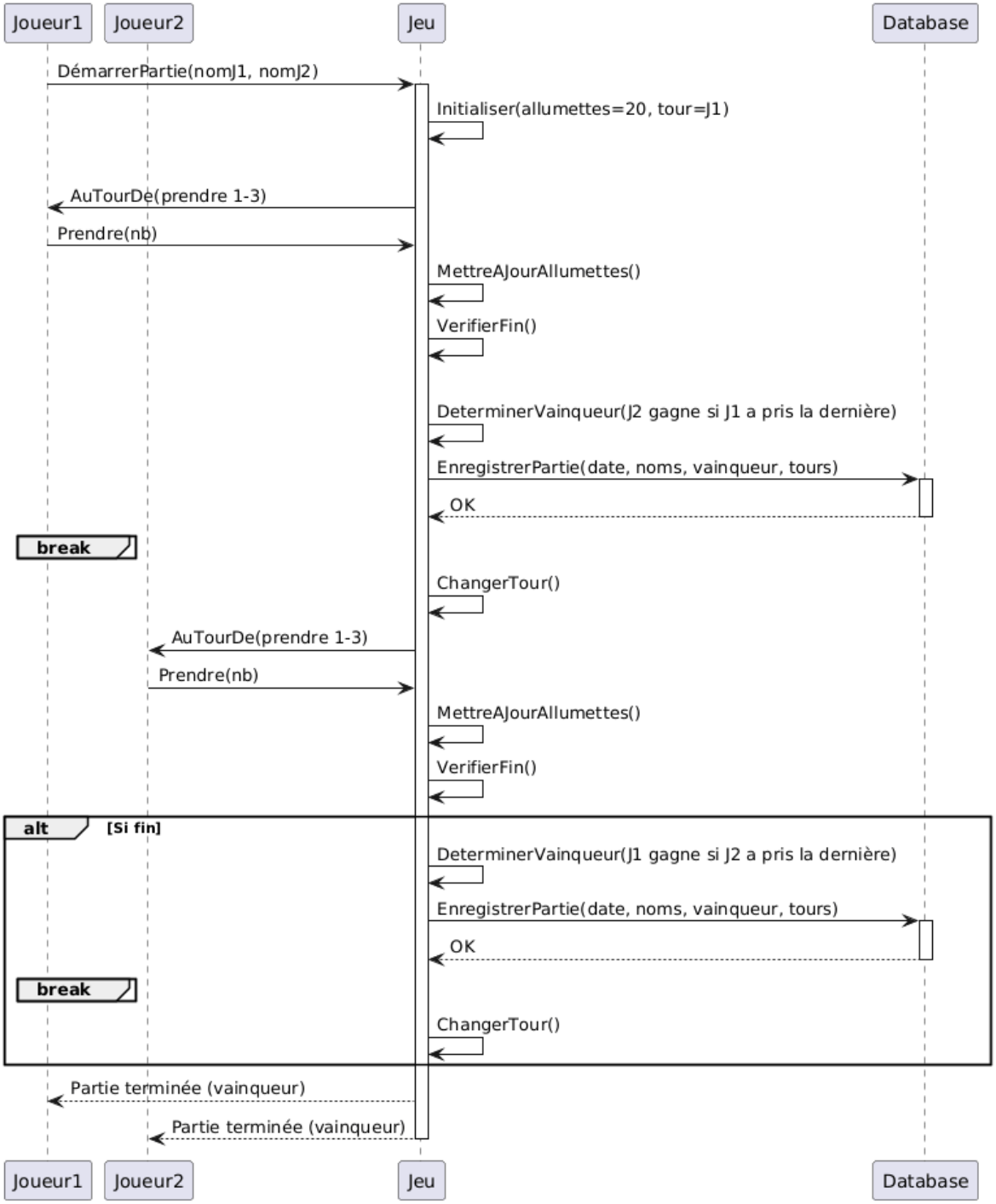
Diagrammes à fournir :

- Diagramme de cas d'utilisation
- Diagramme de classes
- Diagramme de séquence

Outils recommandés : StarUML


Deux joueurs sur le même poste
Démarrer une nouvelle partie
include
Saisir noms des joueurs
Jouer un tour (prendre 1-3 allumettes)
include
Vérifier fin de partie
extend (si fin)
Enregistrer la partie en BD
Joueur
Consulter l'historique des parties
Consulter les scores


Jeu
allumettesRestantes: int = 20
joueurCourant: Joueur
DemarrerPartie(j1: Joueur, j2: Joueur): void
ChangerTour(): void
VerifierFin(): bool
DeterminerVainqueur(): Joueur
DatabaseHelper
EnregistrerPartie(partie: Partie): void
GetHistorique(): List<Partie>
GetScores(): Dictionary<string, int>
1
utilise
2
1
crée
1
1
gère
*
Joueur
nom: string
PrendreAllumettes(nb: int): void
Partie
idPartie: int
datePartie: DateTime
nomJoueur1: string
nomJoueur2: string
vainqueur: string
nombreTours: int


Joueur1
Joueur2
Jeu
Database
DémarrerPartie(nomJ1, nomJ2)
Initialiser(allumettes=20, tour=J1)
AuTourDe(prendre 1-3)
Prendre(nb)
MettreAJourAllumettes()
VerifierFin()
DeterminerVainqueur(J2 gagne si J1 a pris la dernière)
EnregistrerPartie(date, noms, vainqueur, tours)
OK
break
ChangerTour()
AuTourDe(prendre 1-3)
Prendre(nb)
MettreAJourAllumettes()
VerifierFin()
alt
[Si fin]
DeterminerVainqueur(J1 gagne si J2 a pris la dernière)
EnregistrerPartie(date, noms, vainqueur, tours)
OK
break
ChangerTour()
Partie terminée (vainqueur)
Partie terminée (vainqueur)
Joueur1
Joueur2
Jeu
Database